column, second row would be referenced by `C2R1`). Now, when you are storing the values row-oriented (the left-hand side of figure 5.5), you start in the first row and then move on to the second row, and so on, resulting in a storage layout (on disk or in memory) that stores the first row, the second row, and so on, until the last row. In contrast, when storing the values column-oriented (the right-hand side of figure 5.5), you start in the first column and then move on to the second column, and so on, resulting in a storage layout (on disk or in memory) that stores the first column, the second column, and so on, until the last column.

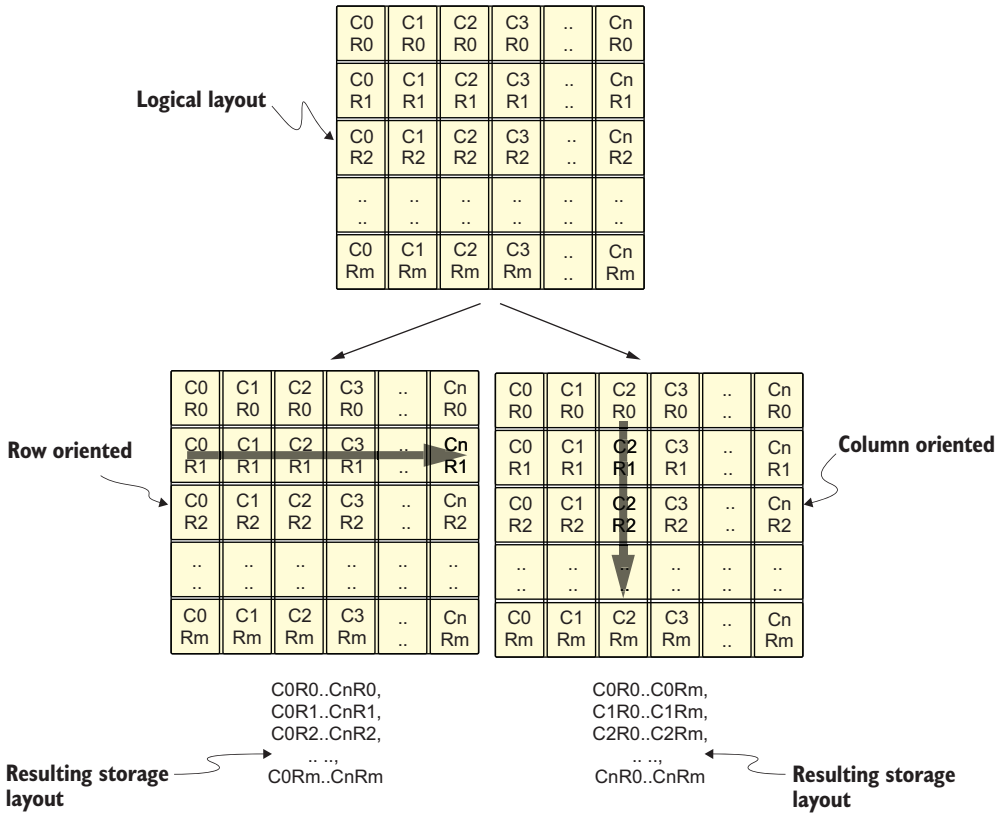


Figure 5.5 Row-oriented vs. column-oriented storage

You may be wondering, why bother with column-oriented storage? It turns out, columnar storage has a number of advantages for OLAP use cases, which includes our observability use cases.

In observability use cases, you can have many dimensions of interest, resulting in high cardinality and many columns. For any given query, there may only be a few columns relevant for a query. Columnar data stores allow for faster and fewer scans, since